The verification type lattice (D25)

Status: applied for the finite verification lattice. Property synthesis is the dual operation and remains planned (Plans B and C below). This note records what is implemented, what it guarantees, and how the larger property-synthesis program sits on top of it.

Why the lattice exists

Before D25 the verification loop had three partly independent descriptions of the same input space:

- `Smt` took `(string * FieldType) list`;
- `DemoDomain` hand-wrote `fields`, `smtFields`, `universe`, and two Hedgehog generators;
- `Refiner.refine` took `fields` and `universe` and synthesized guards from the universe, not from the type.

That is three places to change when a field changes, and it permits the generators, the solver, and the refiner to disagree. D25 makes one value the coordinate system for all three.

The lattice is **a projection of the domain binding** (D9), not a second source of truth. `components/demo/demo/player.scale` remains the one editable specification (D7). The lattice is derived at the host boundary; it is not persisted and it never rewrites the document by itself.

This is the **verification-domain** lattice. It is not the D12 runtime set-theoretic sort record (`(option Name, Extension)`); the bridge between them is deferred and would be Plan C.

The representation

```
type Field =
    {
        Name: string
        Type: DomainExpression.FieldType
        Coordinates: ScaleFile.FieldValue list option
        Narrowing: ScaleFile.RefinementGuard list
    }

type Lattice = { Fields: Field list }
```

- `Coordinates = Some values` is a finite domain. `create` requires at least one value, a value of the field's sort, and (for `IntegerType` or `NaturalType`) a value the QF_LIA encoding can represent exactly (finite, integral, Int32 range).
- `Coordinates = None` is symbolic. Z3 can reason about it, but a finite generator and the exhaustive refiner cannot enumerate it. D56 gives the refiner a witness-guided path for this case; a symbolic field can still feed the solver and still be narrowed. This is not the same as an empty domain, which is refused.
- `Narrowing` is the current region pinned inside the type's domain. It is a conjunction of single-field `ScaleFile.RefinementGuard` values. `applyGuard` adds one; re-applying the same guard is idempotent; a guard that empties a finite field is refused.

The order on regions is the product order: a lattice is narrower when every field's narrowed coordinate set is a subset of the other's. D25 does **not** implement a lattice meet/join algebra or a minimal-region search; the refiner still selects by proved closure and then by coverage, with enumeration order as the tie-break. D56 adds the symbolic case: witness-guided candidate seeds and a bounded weakest-threshold search, still gated by the same proof. That is deliberate (Plan A), not an accident.

Field sorts

`FieldType` is the field's domain sort. It decides three things at once: the SMT sort Z3 reasons about, the values a finite generator may produce, and the F# type the emitter writes.

- `BooleanType` is a truth value. Use it for flags, predicates, and any field whose guards are only `is true` or `is false`. It has no numeric threshold.
- `IntegerType` is an exact, discrete number on the SMT `Int` sort. Use it for counts, indices, versions, retries, and token budgets: quantities where a fractional value is meaningless.
- `NaturalType` is an `IntegerType` constrained to be at least zero. Use it when the same discrete quantity cannot be negative. The non-negativity is a domain axiom, so Z3 can use it in a proof rather than rediscovering it.
- `RealType` is a continuous number on the SMT `Real` sort. Use it for measurements and ratios where fractional bounds matter: positions, speeds, temperatures, probabilities, and normalized scores. Its model value is a finite `double`; do not use it for exact decimal bookkeeping such as money.

The F# emitter is sort-faithful: `RealType` becomes `float`, `IntegerType` and `NaturalType` become `int`, and `BooleanType` becomes `bool`. The SMT theory is `QF_LIRA` when a domain mixes Real and integer fields, `QF_LRA` when it is Real only, and `QF_LIA` otherwise.

The three directions

Z3 reads

`Smt.createFromLattice` declares one constant per field at the lattice's sort. `Smt.precondition` conjoins the sentence's conditions, the document's refinements, and `TypeLattice.constraints`. A lattice guard whose canonical `.scale` text already appears in the document is not conjoined twice; `ScaleFile.guardText` is the identity. The initial demo obligation therefore stays byte-identical, and the refined one stays byte-identical because the document already carries `health > 1`.

`Smt.createDomain` and `Smt.create` still accept a sort-only list. They build `TypeLattice.ofFieldTypes`, whose fields are symbolic and carry no narrowing. Legacy callers, including the decision-table checker, see exactly the previous encoding.

Hedgehog reads

`generatorFor`, `booleanGeneratorFor`, and `numericGeneratorFor` build their generators from `coordinatesAfterNarrowing`. A symbolic field or an emptied finite field is an `Error` **before** a generator is built, so an empty domain is a named refusal rather than Hedgehog `GaveUp`. The typed generators validate the coordinate shapes up front; they do not hide a `failwith` inside a `Gen.map`.

Sampling is uniform over the narrowed coordinates (`Gen.item`). The generator does not infer a range or bias toward zero; the declared coordinates are the domain. A domain that wants a different distribution must change its coordinates or supply its own harness.

`DemoDomain.universe` is now `TypeLattice.universe lattice makePlayer`, so the refiner's exhaustive universe and the generators are two readings of one region. `DemoDomain.fields` and `DemoDomain.smtFields` are projections of the same lattice.

The refiner reads and writes

`Refiner.refine` takes the lattice, the guards the document already carries, and a `make` projection. It folds those guards into the lattice with `TypeLattice.withGuards`. A finite lattice then derives the finite universe, takes candidate guards from `TypeLattice.candidateGuards`, proves closure with Z3, and ranks the survivors by coverage. A symbolic lattice (D56) has no universe: it takes the solver's unrefined counterexample, seeds the weakest guards that exclude it with `TypeLattice.candidateGuardsFromWitness`, widens each numeric seed outward, and bisects to the

weakest proved threshold. Either path writes the chosen guard back with `TypeLattice.applyGuard`; the result is a `RefinementResult` carrying both the candidate and the narrowed lattice. Refinement is therefore a write into the shared coordinate system, not a value that lives only in the document.

On a symbolic lattice `Candidate.Coverage = 0` records that coverage was not measurable; the guard is still chosen only because `pre ∧ guard ∧ ¬post` was proved. A symbolic domain without a counterexample, or with a field the counterexample omits, is a named `Error`; no proved guard is `Ok None`, never a silent pass.

`Refiner.evaluateGuard` delegates to `TypeLattice.guardHoldsValue`, so the runtime evaluator, the refiner, and the lattice have one guard semantics.

Named guards (D41)
A document refinement can name an image-resident guard metaobject (D35): `refinement: <field> guard <name>`. The lattice models this as `ScaleFile.NamedGuard of field * name`. Because a named guard is a quotation and not a typed relation, it has no field type and no SMT encoding of its own.

The lattice evaluates it through `GuardResolver` (`name -> FieldValue -> GuardVerdict`), which the evaluator builds over the image: `EvaluatorRuntimeGuard.resolver` finds the guard by name, runs its quotation in the shared probe, and answers `GuardHolds`, `GuardDoesNotHold`, or `GuardUndecided reason`. The resolver-aware operations are `coordinatesAfterNarrowingWithResolver`, `assignmentsWithResolver`, `universeWithResolver`, `applyGuardWithResolver`, `withGuardsWithResolver`, and the `generator...WithResolver` functions.

The typed-only names remain, and they refuse a named guard by name. So do the SMT encoder and the emitter: a quotation is outside QF_LIA, and writing a second symbolic interpreter would be a second semantics (D10). A named guard can therefore narrow the finite generators and the refiner's coverage, but it cannot be proved; the discharge is `Unavailable`, never a silent omission.

What D25 guarantees

- **Theory alignment is checked before sampling.** A non-integral coordinate or guard threshold is refused by `TypeLattice.create`/`applyGuard` with the same Int32 rule the solver uses. Hedgehog can no longer sample a value Z3 cannot represent.
- **No vacuous finite domain.** `create` and `applyGuard` refuse empty finite regions; the refiner also rejects zero-coverage candidates.
- **No invalid lattice through the public API.** The `Field` and `Lattice` record representations are private. `create`, `ofFieldTypes`, `applyGuard`, and `withGuards` are the public constructors, so a caller cannot bypass the coordinate, sort, and threshold checks by writing a record directly.
- **Incremental re-entry uses the current region.** `withGuards` folds the document's existing refinements into the lattice; `Refiner.refine` does this before building the universe, so coverage and the returned lattice describe the inputs the document already admits rather than a stale pristine region.
- **One field vocabulary.** `DemoDomain.fields`, `DemoDomain.smtFields`, `DemoDomain.universe`, the harness generators, the SMT encoding, and the refiner all derive from `DemoDomain.lattice`.
- **The document stays the source of truth.** The lattice is derived; the refiner's write is still the append to the `.scale` document. The refined encoding reads both, with the document's guard deduplicated.
- **Lattice/document divergence is refused.** Before rendering, `Emitter.emitFSharp` and `Emitter.emitSmt` check that every lattice constraint is carried by the document's refinement text. A narrowing the document lacks is refused, so the F# and SMT artifacts cannot disagree.

- **The pinned artifacts do not move.** The lattice's initial narrowing is empty and the refined narrowing duplicates the document's guard, so the emitted `.smt2` text is unchanged.

Property synthesis is the dual

Generator synthesis reads a region and samples it. Property synthesis reads the same region in the opposite direction:

1. **Forced** — what the type forces (parametricity, subtyping, structure, nominal tags). No testing is needed; these are free theorems and inherited templates.
2. **Permitted** — what the type allows. This is the lattice's coordinate region.
3. **Residual** — permitted but not forced. This is where properties are needed; it is a region difference, not a list of examples.
4. **Vacuous** — empty property domains. Emptiness analysis must reject them before a vacuously true property is proposed.

D25 implements the **permitted** region and the **pinned** region (`Narrowing`). It does not implement forced consequences, the residual difference, or property-domain emptiness as a first-class value. It is the substrate those operations need, not the operations themselves.

The planned loop is:

1. derive forced properties from the lattice's sorts and structure;
2. compute the residual;
3. probe the residual with mutation, differential oracles, metamorphic templates, and bounded SMT search;
4. filter candidates by non-vacuity and by discrimination (a mutant or an oracle disagreement);
5. run survivors through Hedgehog;
6. elaborate with Z3, distinguishing "a counterexample exists" from "the assumption set is too weak";
7. refine the survivor as a given, invariant, or obligation into `.scale`;
8. re-infer, which shifts the lattice and regenerates generators and templates.

The human adjudicates intent-shaped questions only; the type layer collapses the structural, algebraic, and non-vacuous candidates.

The four refinement targets

Target	D25 status	Why
Given / precondition	**implemented** as lattice narrowing	`ScaleFile.Guard` already matches the runtime and SMT vocabularies
Invariant	**not implemented**	needs state, a transition relation, and induction; `SentencePattern.Invariant` means "the verb never holds", not a loop invariant
Theorem obligation / lemma	**not implemented**	`Smt.Outcome` is a four-way verdict; there is no proof log, subgoal, or lemma store, and Z3 does not report missing assumptions
Type refinement	**not implemented**	needs the D12 sort bridge and guards with conjunction/negation/nominal tags; `ScaleFile.Guard` is single-field and positive

Plans and risk

Plan	Scope	Risk	Possibility	Decision
A	Finite coordinate lattice shared by Z3, Hedgehog, and the refiner; narrowing write-back; theory-aligned validation	**low** — additive, pinned artifacts unchanged, no new solver theory	**high** — closes the documented `FieldType`/sort-record seam at the verification boundary	**implemented**
B	Region algebra (subsumption/difference), residual computation, type-indexed templates, mutation/oracle candidates, non-vacuity certificates	**medium** — needs a property-candidate type, a mutation boundary, and a minimal-region search; the current coverage heuristic must not be mistaken for minimality	**medium** — each piece is independently testable, but the property hypothesis space can grow faster than the suite	not now
C	Full four-target CEGIS over D12 set-theoretic sorts: state invariants and induction, lemma/subgoal store, nominal tags, axiom context, structural/nominal property knob	**high** — widens the theory, changes the bundle contract, and risks unsoundness if forced/permitted/vacuous are conflated	**lower** — requires a new decision record and a proof-obligation representation the current fragment does not have	not now

Plan A was chosen because it is the smallest change that makes the three readers agree without changing the trust boundary. It also gives Plan B a stable coordinate system: property synthesis can index templates and residuals by the same fields, sorts, coordinates, and narrowings the generators already use.

Deliberately out of scope for D25

- A lattice-theoretic minimality proof. The refiner remains proved-closure-plus-coverage, not "least region strictly stronger than the current one".
- Conjunction/negation/nominal guard forms. `Narrowing` is a list of existing single-field guards.
- Mutation-driven property inference and property templates. No property candidate type exists yet.
- Invariants, lemmas, proof logs, and induction. `Smt.Outcome` gained `Witnessed`/`NoWitness` for existential `some` (D33); the universal path is unchanged.
- The D12 runtime sort record. The verification lattice is a separate, finite, host-bound projection.
- Persisting the lattice. The `.scale` document remains the only editable truth.